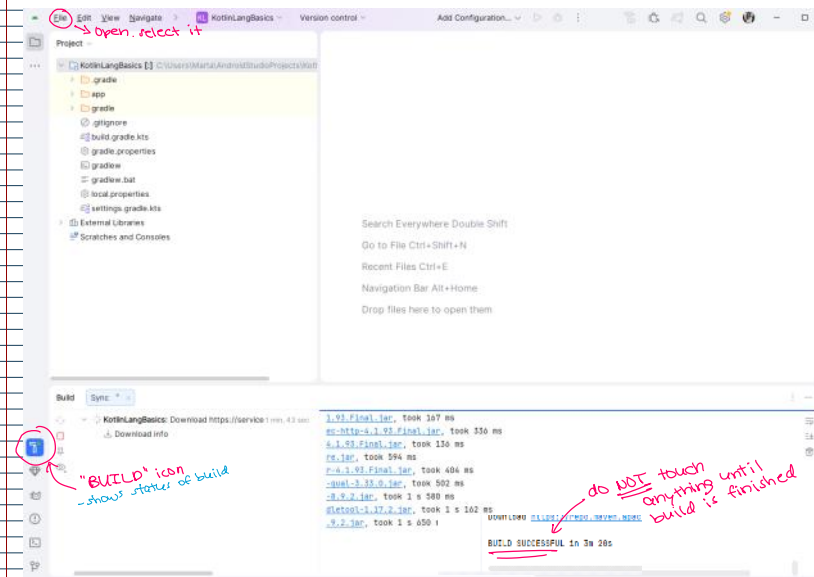


Example code - Kotlin Basics

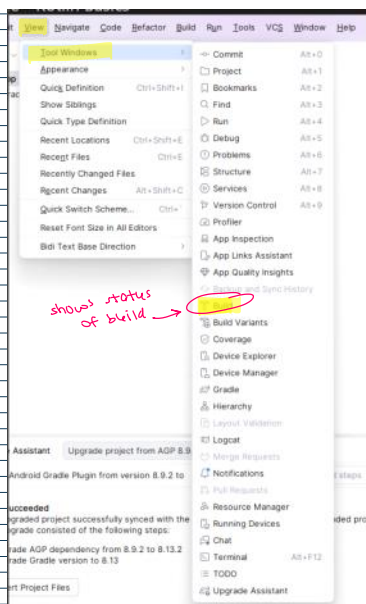
May 19, 2026 4:28 PM

Extracted this folder into 'C:/Users/Marta/AndroidStudioProjects' and opened it from there from within the IDE
KotlinLan...

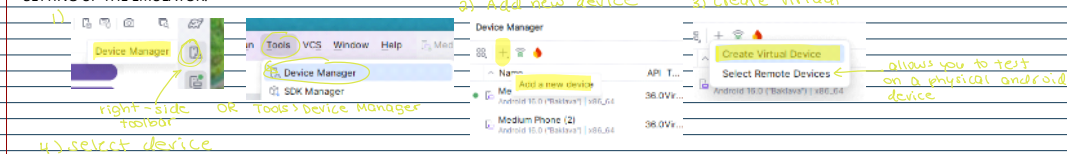
OPENING A PROJECT:

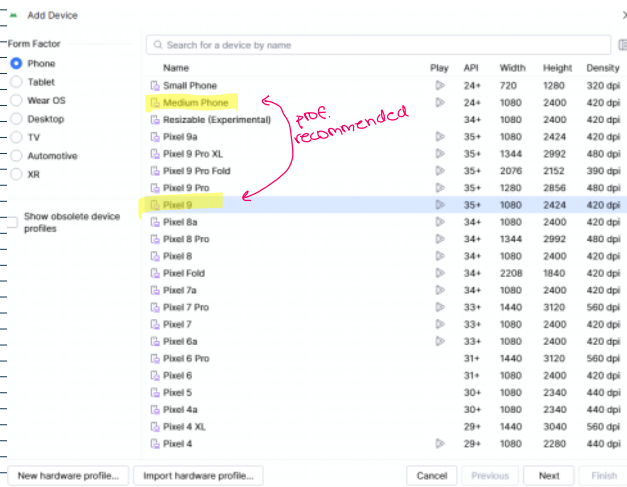


Tools found under 'View'

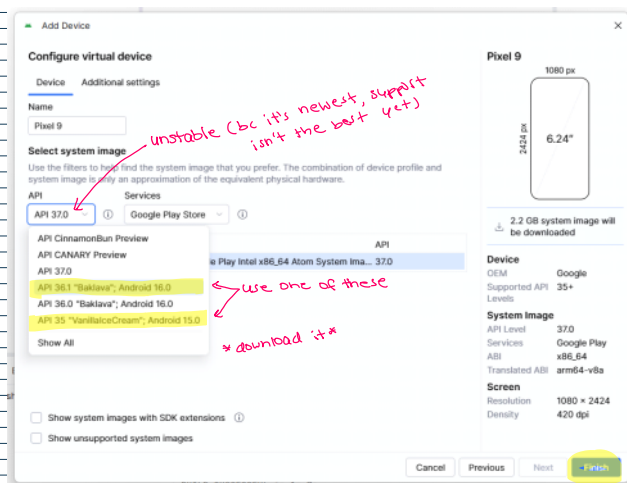


SETTING UP THE EMULATOR:

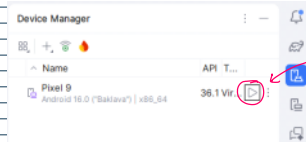




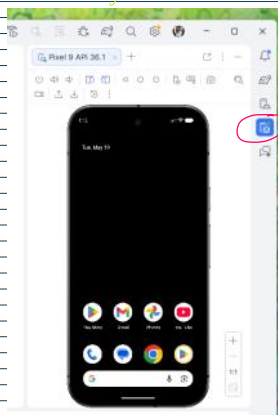
5) Configure name + API



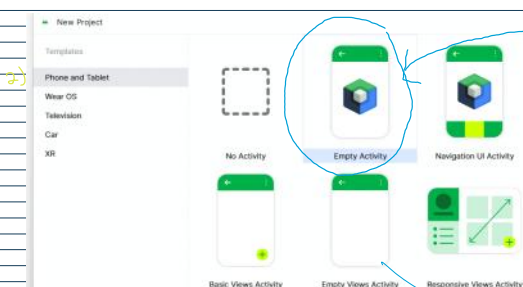
6) Start emulator.



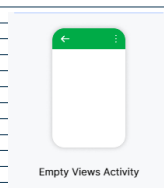
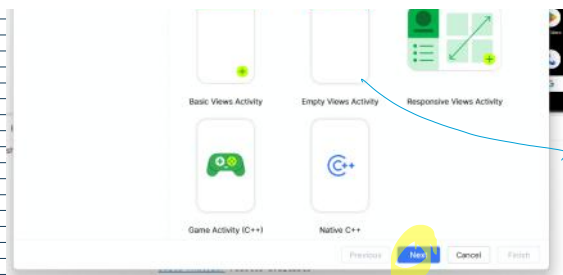
7) Go to "Running devices" tab



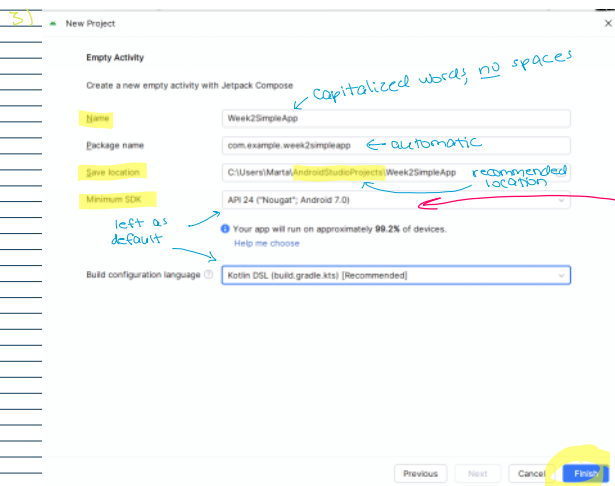
CREATING NEW PROJECT → File > New > New Project



Makes a JetPack Compose App
- What we should use

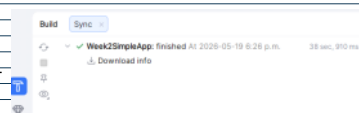


<- Traditional approach, not JetpackCompose
 - Will have xml file with lots of html
 - Gives you the option to use Java instead of Kotlin
 - we use xml + draw & draw for UI design



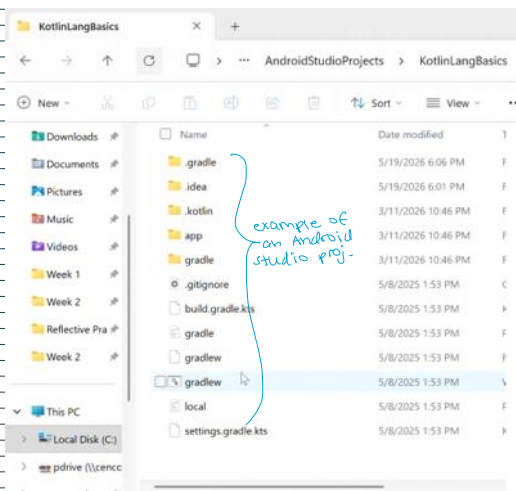
Minimum SDK = level of support according to device/Android version
 - Don't go too high bc support isn't fully fledged out yet for newer versions

- Make sure build is finished before touching anything



- Then click Run arrow to make sure code is working

FILE STRUCTURE in system:



↳ In Android Developer:

Android

- app
 - manifests
 - kotlin+java ← *code*
 - com.example.week2simpleapp ← *1st package is the one we work in*
 - ui.theme
 - Color.kt
 - Theme.kt
 - Type.kt
 - MainActivity.kt ← *main code file!*
 - com.example.week2simpleapp (androidT) ← *2*
 - com.example.week2simpleapp (test) ← *3*
 - res
 - Gradle Scripts

specific main activity class for res, party connection

UI design related files

important scripts

- build the app
- define dependencies

res

- drawable
- mipmap
- values
 - colors.xml
 - strings.xml
 - themes.xml
- xml
- Gradle Scripts

* AndroidManifest.xml - important!

AndroidManifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.week2simpleapp">
    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Week2SimpleApp">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.Week2SimpleApp">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
  
```

intent - filter

This filter defines the *home/main screen* when there are multiple

app

- manifests
 - AndroidManifest.xml

- VERY IMPORTANT!

- For external database, API, Etc connectivity
- Gives permissions to do so

Gradle folder

Gradle Scripts

- build.gradle.kts (Project: Week2SimpleApp)
- build.gradle.kts (Module :app) ← *App meta-data, dependencies used, can also add new dependencies*
- proguard-rules.pro (ProGuard Rules for ":app")
- gradle.properties (Project Properties)
- gradle-wrapper.properties (Gradle Version)
- libs.versions.toml (Version Catalog "libs")
- local.properties (SDK Location)
- settings.gradle.kts (Project Settings)

MAIN ACTIVITY FILE - *main code file*

```

1 package com.example.week2simpleapp
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.material3.Scaffold
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.tooling.preview.Preview
14 import com.example.week2simpleapp.ui.theme.Week2SimpleAppTheme
15
16 class MainActivity : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         enableEdgeToEdge()
20         setContent {
21             Week2SimpleAppTheme {
22                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
23                     Greeting(
24                         name = "Android",
25                         modifier = Modifier.padding(paddingValues = innerPadding)
26                     )
27                 }
28             }
29         }
30     }
31 }
32
33 2 Usages
34 @Composable
35 fun Greeting(name: String, modifier: Modifier = Modifier) {
36     Text(
37         text = "Hello $name!",
38         modifier = modifier
39     )
40 }
41
42 @Preview(showBackground = true)
43 @Composable
44 fun GreetingPreview() {
45     Week2SimpleAppTheme {
46         Greeting(name = "Android")
47     }
48 }

```

lifecycle method
- NEED for code

main() code here

- this allows for modularity

2:13
2:21

Linking a button to another page:

```

Button(
    onClick = {
        // creating a button that leads to another kotlin file (SecondActivity)
        val intent = Intent(packageContext = this@MainActivity, cls = SecondActivity::class.java)
        startActivity(intent)
        //Toast.makeText(this@MainActivity, "Button Clicked", Toast.LENGTH_LONG).show()
    }
)

```

current screen

screen you want to link to (kotlin file)

current activity

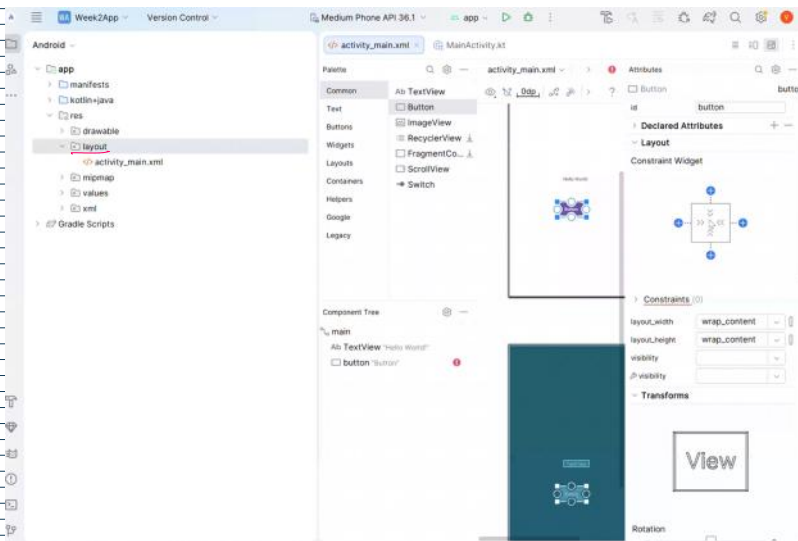
next activity/screen



→ outdated, used before JetPack compose became a thing.

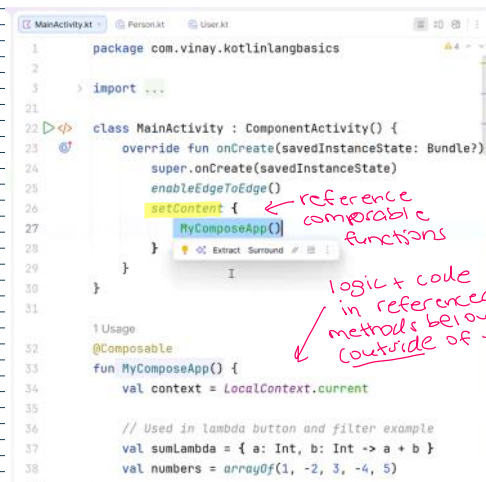
- use xml only for UI

Example of a NON-JetPack Compose App (Empty Views Activity)



← drag & drop
+ XML

vs. kotlin → everything can be in one file
(design + logic)



Ideally: reference functions in setContent
- Do not code directly in setContent

MainActivity.kt

```

1 package com.vinay.kotlinlangbasics
2
3 > import ...
4
21
22 1 Usage
23 class MainActivity : ComponentActivity() {
24     override fun onCreate(savedInstanceState: Bundle?) {
25         super.onCreate(savedInstanceState)
26         enableEdgeToEdge()
27         setContent {
28             MyComposeApp() // reference composable functions here, code in the functions themselves
29         }
30     }
31 }
32
33 1 Usage
34 @Composable
35 fun MyComposeApp() {
36     val context = LocalContext.current
37
38     // Used in lambda button and filter example
39     val sumLambda = { a: Int, b: Int -> a + b }
40     val numbers = arrayOf(1, -2, 3, -4, 5)
41
42     Column(modifier = Modifier.padding( all = 16.dp)) {
43
44         // the below are controls + associated logic. Not css
45
46         // 1st button
47         Button(onClick = {
48             val sum = sum( a = 5, b = 6)
49             Toast.makeText(context, text = "sum=$sum", duration = Toast.LENGTH_SHORT).show()
50             printSum1(context, a = 7, b = 8)
51         }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
52             Text( text = "Functions")
53         }
54
55         // 2nd Button
56         Button(onClick = {
57             val numberList = listOf(1, 2, 3, 4, 5, 6)
58             numberList.forEach {
59                 Toast.makeText(context, text = "number=$it", duration = Toast.LENGTH_SHORT).show()
60             }
61         }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
62             Text( text = "Loops")
63         }
64
65         // 3rd
66         Button(onClick = {
67             val str = describe( obj = "some string")
68             Toast.makeText(context, text = "description=$str", duration = Toast.LENGTH_SHORT).show()
69         }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
70             Text( text = "When Expression")
71         }
72     }
73 }

```

7:30

Functions

Loops

When Expression

Lambda

It Lambda

Kotlin Class

Data Class

```

68 // 4th
69 Button(onClick = {
70     val result = sumLambda(9, 3)
71     Toast.makeText(context, text = "sum from lambda=$result", duration = Toast.LENGTH_SHORT).show()
72 }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
73     Text(text = "Lambda")
74 }
75 // 5th
76 Button(onClick = {
77     val positives = numbers.filter { it > 0 }
78     positives.forEach {
79         Toast.makeText(context, text = "number = $it", duration = Toast.LENGTH_SHORT).show()
80     }
81 }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
82     Text(text = "It Lambda")
83 }
84 // 6th
85 Button(onClick = {
86     val person = Person( firstName = "John", lastName = "Stone", age = 27)
87     Toast.makeText(context, text = "person = ${person.upperCaseFirstName}, ${person.upperCaseLastName}, ${person.age}", duration = Toast.LENGTH_SHORT).show()
88 }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
89     Text(text = "Kotlin Class")
90 }
91 // 7th
92 Button(onClick = {
93     val user = User( name = "john stone", age = 27)
94     Toast.makeText(context, text = "user = ${user.name}, ${user.age}", duration = Toast.LENGTH_SHORT).show()
95 }, modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
96     Text(text = "Data Class")
97 }
98 }
99 }
100
101 // Functions
102 1 Usage
103 fun sum(a: Int, b: Int): Int = a + b
104
105 1 Usage
106 fun printSum1(context: android.content.Context, a: Int, b: Int) {
107     Toast.makeText(context, text = "sum of $a and $b is ${a + b}", duration = Toast.LENGTH_SHORT).show()
108 }
109
110 1 Usage
111 fun describe(obj: Any): String =
112     when (obj) {
113         1 -> "One"
114         "Hello" -> "Greeting"
115         is Long -> "Long"
116         !is String -> "Not a string"
117         else -> "Unknown"
118     }
119 }

```

Introducing Life Cycle methods:


```
MainActivity.kt
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        setContentView {
            LifecycleTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    modifier = Modifier.padding(paddingValues = innerPadding)
                }
            }
        }
    }

    override fun onStart() {
        super.onStart()
        Log.d(tag = "Lifecycle", msg = "Activity -> onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(tag = "Lifecycle", msg = "Activity -> onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(tag = "Lifecycle", msg = "Activity -> onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d(tag = "Lifecycle", msg = "Activity -> onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
    }
}
```

Annotations in the image:

- `onCreate`: most common
- `onStart`: start app
- `onResume`: background process
- `onPause`: pressed back button
- `onStop`: app stops
- `onDestroy`: when app terminates

can view the log to see when lifecycles are triggered:

LOG CAT

The Logcat window shows the following log entries (relevant ones highlighted in yellow in the original image):

- 2026-05-19 19:35:53.571 18898-18898 ashmen com.example.lifecycle
- 2026-05-19 19:35:53.603 18898-18898 CompatChangeReporter com.example.lifecycle
- 2026-05-19 19:35:53.603 18898-18898 DesktopModeFlags com.example.lifecycle
- 2026-05-19 19:35:53.887 18898-18898 Lifecycle com.example.lifecycle
- 2026-05-19 19:35:53.903 18898-18898 Lifecycle com.example.lifecycle
- 2026-05-19 19:35:53.927 18898-18898 HWUI com.example.lifecycle
- 2026-05-19 19:35:53.967 18898-18898 WindowOnBackDispatcher com.example.lifecycle
- 2026-05-19 19:35:55.120 18898-18898 ample.Lifecycle com.example.lifecycle

On the right, the expanded log entry for `Activity -> onStart` is visible, showing the following messages:

- `Activity -> onStart` (highlighted in yellow)
- `Activity -> onResume` (highlighted in yellow)

Annotation: can see the life-cycle states that were triggered